

■ ЛЕКЦИЯ 3 · ИНСТРУМЕНТЫ РАЗРАБОТКИ



Git: ветвление и воркфлоу

Ветки, слияние и rebase, разрешение конфликтов, модели работы в команде и Pull Request — как ведут разработку профессионально

Чернов Евгений

Магистратура «Искусственный интеллект», НИУ ВШЭ

План занятия

1

Ветки

дешёвые указатели, создание и переключение

2

Слияние

fast-forward и трёхсторонний merge

3

Rebase

перенос коммитов и золотое правило

4

Конфликты

почему возникают и как разрешать

5

Воркфлоу

Git Flow, GitHub Flow, trunk-based

6

Pull Request

ревью, защита веток, спасательный круг

Зачем нужны ветки?

Реальная разработка идёт параллельно: несколько задач, эксперименты, исправления — и всё это не должно ломать рабочий код в `main`.

1

Изоляция

эксперимент в ветке не трогает
стабильный `main`

2

Параллельность

каждый работает в своей ветке
одновременно

3

Обзор

готовую ветку удобно
проверить перед слиянием

Создать и переключить ветку

● ● ● терминал

```
$ git branch feature/login
# создали указатель на текущий коммит

$ git switch feature/login
# HEAD теперь указывает на ветку

$ git switch -c feature/login
# создать и сразу переключиться
```

git branch <имя>

создать ветку (указатель)

git switch <имя>

перейти в ветку

git switch -c <имя>

создать и перейти сразу

git branch

список веток; * — текущая

Как слить ветку в main

терминал

```
$ git switch main
$ git merge feature/login
Updating alb2c3d..e4f5g6h
Fast-forward
 src/auth.py | 12 ++++++++

# или с merge-коммитом:
$ git merge --no-ff feature/login
```

1. `switch main`

перейти в ветку-приёмник

2. `merge <ветка>`

вливать изменения из ветки

`--no-ff`

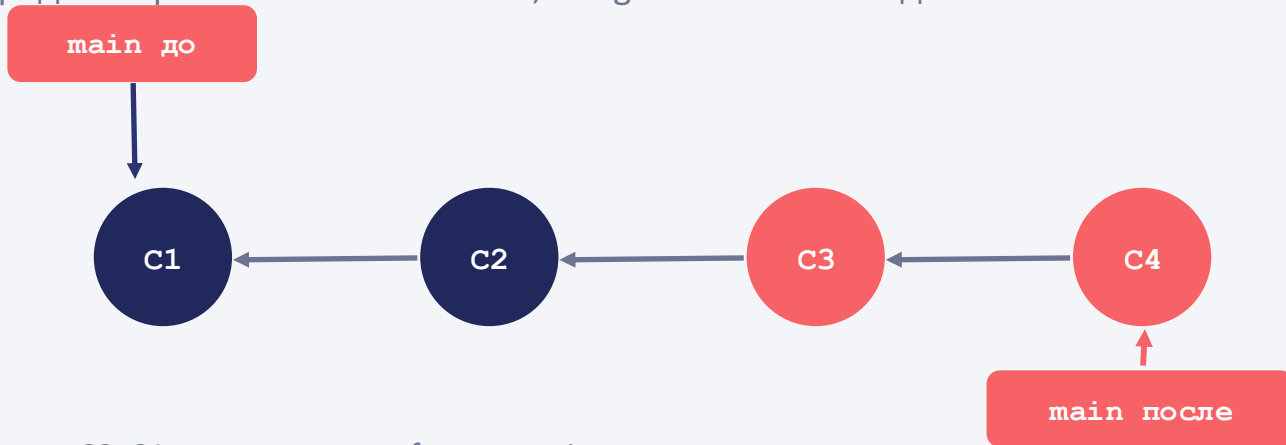
всегда создавать merge-коммит

`--abort`

отменить незавершённое слияние

Fast-forward: просто сдвинуть указатель

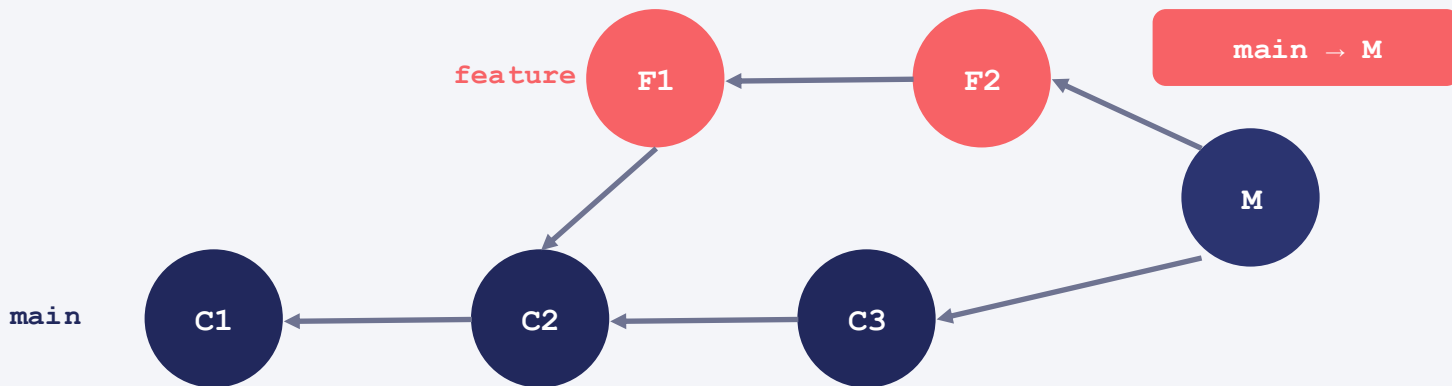
Если в `main` не было новых коммитов, слияние ветки — это просто перемещение указателя `main` вперёд. История остаётся линейной, `merge`-коммит не создаётся.



коммиты C3, C4 сделаны в ветке `feature`; `main` просто «догоняет» их

Трёхсторонний merge: когда ветки разошлись

Если и в `main`, и в ветке были новые коммиты, `Git` строит новый `merge`-коммит с двумя родителями, объединяя обе линии. История ветвления сохраняется.



merge-коммит М связывает C3 (main) и F2 (feature) — у него два родителя

02

■ РАЗДЕЛ

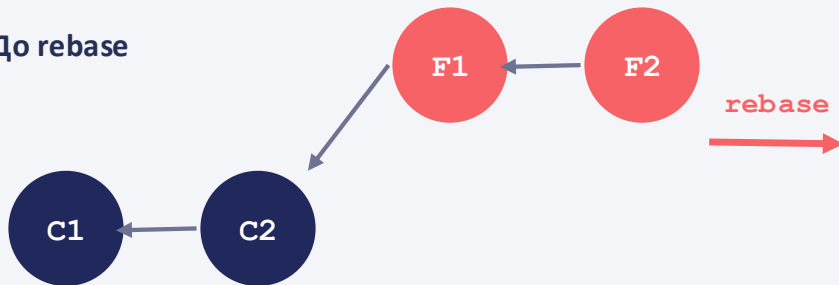
Merge или Rebase?

Два способа объединить работу — и почему rebase нельзя применять к общей истории

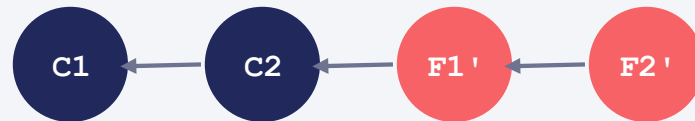
Rebase переписывает коммиты на новую базу

Rebase берёт ваши коммиты и «переносит» их поверх свежего main — создавая их копии. История становится линейной, но коммиты получают новые хеши.

До rebase



После rebase



F1', F2' — новые коммиты с новыми хешами (копии)

Merge против Rebase

merge

- сохраняет реальную историю
- не меняет существующие коммиты
- история нелинейная (много веток)
- безопасен для общих веток

rebase

- линейная, чистая история
- переписывает коммиты (новые хеши)
- удобно перед вливанием ветки
- опасен для уже опубликованных коммитов



Золотое правило rebase: никогда не делайте rebase коммитов, которые уже видели другие (в общей ветке). Переписав общую историю, вы сломаете её всем.

Интерактивный rebase: причёсываем историю

`git rebase -i` позволяет переписать несколько последних коммитов: объединить, переименовать, переставить или удалить — перед тем как показать ветку другим.

 план переписывания


```
$ git rebase -i HEAD~3  
pick    a1b2c3 add login form  
squash  e4f5g6 fix typo  
reword  7h8i9j add validation  
# редактор откроется — меняем  
# действия слева от коммитов
```

pick

оставить коммит как есть

squashслить с предыдущим
коммитом**reword**изменить сообщение
коммита**drop**удалить коммит из
истории

Почему возникает конфликт слияния

Конфликт возникает, когда две ветки изменили одни и те же строки одного файла по-разному. Git не может решить за вас, какой вариант верный, — и просит выбрать.

ветка **main**

```
port = 8080
```

ветка **feature**

```
port = 3000
```



Обе ветки поменяли одну строку config.py. При слиянии Git остановится **и пометит конфликт** — дальше решаете вы: оставить 8080, 3000 или что-то третье.

Как читать и разрешать конфликт

config.py с конфликтом

<<<<<< HEAD

port = 8080

=====

port = 3000

>>>>>> feature

удалите маркеры, оставьте

верный вариант, затем commit

1

Найти — Git помечает файлы как conflicted

2

Открыть — между <<<<< и >>>>> — два варианта

3

Выбрать — оставить нужное, убрать маркеры

4

Зафиксировать — git add + git commit

Модели ветвления

GitHub Flow

main всегда рабочий; фича-ветка → PR → merge

когда: простые проекты, CI/CD

Git Flow

develop, release, hotfix, feature
— строгие правила

когда: релизы по расписанию

Trunk-based

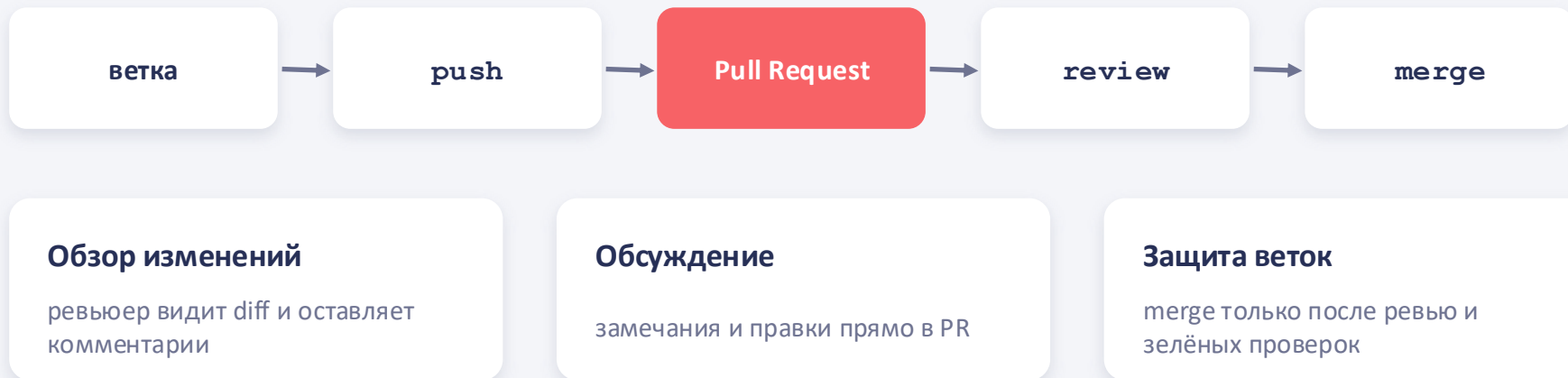
почти всё в main, крошечные ветки, флаги фич

когда: большие команды, быстрый темп

Для этого курса — GitHub Flow: ветка → Pull Request → проверка → слияние.

Pull Request и code review

Pull Request (PR) — это предложение влить вашу ветку в main. До слияния команда видит изменения, обсуждает и проверяет их.



Спасательный круг: откаты и восстановление

git reset --soft

откатить коммит, изменения оставить в индексе

git reset --hard

откатить коммит и удалить изменения (осторожно!)

git revert

создать коммит, отменяющий другой (безопасно)

git reflog

журнал перемещений HEAD — найти «потерянное»

git stash

временно спрятать изменения и вернуться к ним позже

git cherry-pick

перенести отдельный коммит в текущую ветку

Хорошие практики веток и коммитов

branch

Имена веток

feature/login, fix/crash —
префикс + суть, латиницей

commit

Атомарные коммиты

один коммит = одно логическое
изменение

message

Сообщения

повелительно и по делу: «add
auth», не «фиксы»



Чистая история — это забота о команде: по понятным коммитам и веткам легко
делать ревью, искать баги и откатывать изменения.

Что важно унести

1

Ветка — это указатель

создавать и переключать ветки дёшево и быстро

2

Merge vs Rebase

merge сохраняет историю, rebase её выпрямляет

3

Правило rebase

не переписывайте общую, уже опубликованную историю

4

Работа через PR

ветка → Pull Request → ревью → слияние